

Part 2: The Chemical Master Equation (CME)

Define $P_t(n_A)$ = probability of being in state n_A at time t .

The CME (Eq. 5 in paper) says: probability flows in when a reaction brings you to n_A , flows out when a reaction takes you away:

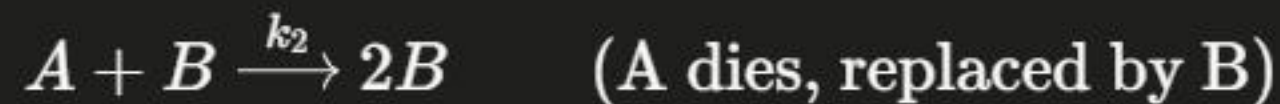
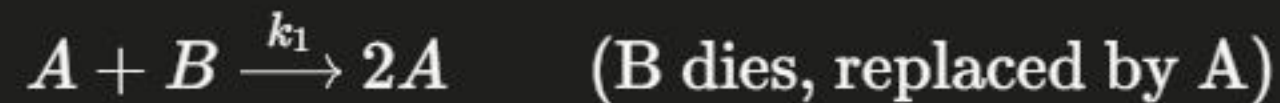
$$\begin{aligned} \frac{d}{dt}P_t(n_A) = & \underbrace{k_1 h(n_A - 1) P_t(n_A - 1)}_{\text{gain: reaction 1 from } n_A - 1} + \underbrace{k_2 h(n_A + 1) P_t(n_A + 1)}_{\text{gain: reaction 2 from } n_A + 1} \\ & - \underbrace{[k_1 + k_2] h(n_A) P_t(n_A)}_{\text{loss: either reaction leaves } n_A} \end{aligned}$$

This is an infinite system of coupled ODEs — one per state. For $N = 100$ that's 101 equations. Analytically intractable in general. That's why we need the control framework to avoid solving it directly.

Part 1: What is the Moran Process?

You have N individuals. Each is either type A or type B . State = n_A (number of A's). So $n_B = N - n_A$.

At each instant, one individual dies and is replaced. The two reactions are:



Mass action propensity — how fast each reaction fires:

$$h(n_A) = \frac{n_A \cdot n_B}{N} = \frac{n_A(N - n_A)}{N}$$

Why this form? You need one A and one B to meet. Probability of picking an A is n_A/N , probability of picking a B is n_B/N , scaled by system size N . Same propensity for both reactions — only the rate constants k_1, k_2 differ.

Absorbing states: $n_A = 0$ (all B) and $n_A = N$ (all A). Once you hit either, you stay forever. This is extinction — the key biological feature the paper controls.

Selective advantage: $\gamma = k_1^0/k_2^0$. If $\gamma > 1$, A has advantage — system drifts toward $n_A = N$ faster.

Part 3: The Optimal Control Problem

We want to keep n_A away from the absorbing boundaries as long as possible. The controller adjusts $k_1(t)$, $k_2(t)$ in real time based on current state.

Utility: how long before extinction:

$$U = \beta T_{\text{exit}} - C(\text{control effort used})$$

We want to **maximize** this — maximize exit time while penalizing how much we deviate from the natural rates k_1^0 , k_2^0 .

Formally (Eq. 7):

$$V(n, \beta) = \max_{k(\cdot)} \mathbb{E}_{k(\cdot)} [\beta T_{\text{exit}} - C(n(\cdot), k(\cdot))]$$

β is a tuning knob — larger β means "I care more about staying alive longer, willing to pay more cost."

Part 4: Why KL Divergence as Control Cost?

The control cost needs to satisfy three physical properties:

1. Decomposability: total cost = sum over reactions. Each reaction controlled independently:

$$C = \sum_r D_r(j_r, j_r^0)$$

2. Non-negativity: $k_r \geq 0$ always (can't have negative reaction rates). The cost function must enforce this naturally.

3. Positive homogeneity: if you scale both controlled and uncontrolled fluxes by ρ , cost scales by ρ too.

The paper proves (Appendix B) that **f-divergence is the only class** satisfying all three. The KL choice $f(x) = x \log x - x + 1$ gives:

$$C_{\text{KL}}(n, k) = \sum_r h_r(n) \left(k_r \log \frac{k_r}{k_r^0} - k_r + k_r^0 \right)$$

Why KL specifically? Because $f(x) = x \log x - x + 1$ has a special property: its Legendre transform is the exponential function. This is what makes everything tractable. Let's see how.

Part 5: The HJB Equation and Why It's Hard

Standard dynamic programming gives the Hamilton-Jacobi-Bellman equation (Eq. 12):

$$-\frac{\partial}{\partial t}V_t(n, \beta) = \beta U_t(n) + \max_{k \geq 0} [L_k V_t(n, \beta) - C(n, k)]$$

where L_k is the **backward generator** — it tells you how the value function changes due to transitions:

$$L_k f(n_A) = k_1 h(n_A) [f(n_A + 1) - f(n_A)] + k_2 h(n_A) [f(n_A - 1) - f(n_A)]$$

This is a **nonlinear PDE** (because of the max over k). On a state space of size $N + 1$, it's a system of nonlinear ODEs. Generally intractable.

Dynamic Programming: The One Idea Behind Everything

The core insight is called **Bellman's principle of optimality**. It says:

Whatever your current state is, the optimal strategy from here onward is independent of how you got here.

In other words — **the future doesn't care about the past, only about the present state.**

This lets you break the problem into tiny pieces. Instead of optimizing over the entire future trajectory $k(t), k(t + dt), k(t + 2dt), \dots$, you just ask:

"What's the best thing to do in the next tiny interval $[t, t + dt]$, and then optimally from there?"

Mathematically, consider a tiny time interval dt . Split the total reward into:

$$V_t(n, \beta) = \max_k \left[\underbrace{\text{reward in } [t, t + dt]}_{\text{you control this}} + \underbrace{V_{t+dt}(n', \beta)}_{\text{future: already optimal by assumption}} \right]$$

where n' is whatever state you land in after time dt .

This is the **dynamic programming principle** — and the HJB equation is just what you get when you take $dt \rightarrow 0$.



Writing out the expectation

Taking the expectation over these three cases:

$$\begin{aligned} V_t(n, \beta) = \max_k \bigg[& \underbrace{(1 - (k_1 + k_2)h \, dt)}_{\text{no jump}} \underbrace{\left(-C(n, k) \, dt + V_{t+dt}(n, \beta)\right)}_{\text{cost}} \\ & + \underbrace{k_1 h \, dt}_{\text{reaction 1}} \left(-C(n, k) \, dt + V_{t+dt}(n + 1, \beta)\right) \\ & + \underbrace{k_2 h \, dt}_{\text{reaction 2}} \left(-C(n, k) \, dt + V_{t+dt}(n - 1, \beta)\right) \bigg] \end{aligned}$$

Now expand and drop all terms of order dt^2 (like $dt \cdot dt$, which vanish):

$$\begin{aligned} V_t(n, \beta) = \max_k \bigg[& -C(n, k) \, dt + V_{t+dt}(n, \beta) \\ & + k_1 h \, dt [V_{t+dt}(n + 1, \beta) - V_{t+dt}(n, \beta)] \\ & + k_2 h \, dt [V_{t+dt}(n - 1, \beta) - V_{t+dt}(n, \beta)] \bigg] \end{aligned}$$

Taking $dt \rightarrow 0$: where the HJB comes from

Rearrange — move $V_{t+dt}(n, \beta)$ to the left:

$$V_t(n, \beta) - V_{t+dt}(n, \beta) = \max_k \left[-C(n, k) dt + k_1 h dt [V_{t+dt}(n+1) - V_{t+dt}(n)] + k_2 h dt [V_{t+dt}(n-1) - V_{t+dt}(n)] \right] + \beta U_t(n, \beta)$$

Divide both sides by dt and take $dt \rightarrow 0$:

$$-\frac{\partial V_t}{\partial t} = \max_{k \geq 0} \left[\underbrace{k_1 h(n) [V_t(n+1) - V_t(n)] + k_2 h(n) [V_t(n-1) - V_t(n)]}_{L_k V_t(n, \beta)} - C(n, k) \right] + \beta U_t(n, \beta)$$

And that's it. That's the HJB. Nothing was assumed — it falls out of taking the limit of tiny time steps.

What is L_k ? Physical meaning.

$$L_k f(n_A) = k_1 h(n_A) \underbrace{[f(n_A + 1) - f(n_A)]}_{\text{change if reaction 1 fires}} + k_2 h(n_A) \underbrace{[f(n_A - 1) - f(n_A)]}_{\text{change if reaction 2 fires}}$$

$L_k f(n_A)$ answers: "How fast is the quantity f expected to change right now, given I'm in state n_A ?"

Think of f as any function of the state — like value, or population count, or anything.

The term $k_1 h(n_A)$ is the **rate** at which reaction 1 fires. If it fires, f changes by $f(n_A + 1) - f(n_A)$. So the expected rate of change due to reaction 1 is $k_1 h \cdot [f(n_A + 1) - f(n_A)]$. Add both reactions: that's $L_k f$.

This is called the **infinitesimal generator** of the Markov process — it generates the dynamics of expected values.

Concrete example: let $f(n_A) = n_A$ (just the population count).

$$L_k n_A = k_1 h(n_A) \underbrace{[n_A + 1 - n_A]}_{=1} + k_2 h(n_A) \underbrace{[n_A - 1 - n_A]}_{=-1} = (k_1 - k_2) h(n_A)$$

This says: the expected rate of change of population = $(k_1 - k_2) \cdot h(n_A)$. Makes sense — if $k_1 > k_2$, population drifts up on average.

Why is it nonlinear?

Look at the HJB again:

$$-\frac{\partial V_t}{\partial t} = \beta U_t(n) + \max_{k \geq 0} \left[L_k V_t - C(n, k) \right]$$

The max over k is what makes this nonlinear. The quantity inside depends on k in two ways:

- Through $L_k V_t$: linearly in k (since $L_k f = k_1 h \Delta_+ f + k_2 h \Delta_- f$)
- Through $-C(n, k)$: nonlinearly in k (since C_{KL} involves $k \log k$)

When you optimize k and plug the result back in, you get a complicated nonlinear expression in V_t . Specifically with KL cost you get $\exp(\nabla V)$ terms, which makes the whole thing nonlinear.

Before Cole-Hopf: nonlinear in V .

After Cole-Hopf ($Z = e^V$): linear in Z . Miracle.

Step 6: Optimizing Over k Inside the HJB

We have the HJB:

$$-\frac{\partial V_t}{\partial t} = \beta U_t(n) + \max_{k \geq 0} \left[\underbrace{k_1 h \nabla_+ V + k_2 h \nabla_- V}_{L_k V} - C(n, k) \right]$$

where I'm using shorthand:

$$\nabla_+ V := V(n_A + 1) - V(n_A) \quad \nabla_- V := V(n_A - 1) - V(n_A)$$

The max is over both k_1 and k_2 simultaneously. Since the two reactions are **independent** (each has its own cost, no coupling), we can optimize them **separately**:

$$\max_{k_1, k_2 \geq 0} [\dots] = \max_{k_1 \geq 0} [k_1 h \nabla_+ V - C_1] + \max_{k_2 \geq 0} [k_2 h \nabla_- V - C_2]$$

So focus on one reaction. The problem for reaction 1 is:

$$\max_{k_1 \geq 0} \left[k_1 h \nabla_+ V - C_{\text{KL}}(k_1) \right]$$

Step 6: Optimizing Over k Inside the HJB

We have the HJB:

$$-\frac{\partial V_t}{\partial t} = \beta U_t(n) + \max_{k \geq 0} \left[\underbrace{k_1 h \nabla_+ V + k_2 h \nabla_- V}_{L_k V} - C(n, k) \right]$$

where I'm using shorthand:

$$\nabla_+ V := V(n_A + 1) - V(n_A) \quad \nabla_- V := V(n_A - 1) - V(n_A)$$

The max is over both k_1 and k_2 simultaneously. Since the two reactions are **independent** (each has its own cost, no coupling), we can optimize them **separately**:

$$\max_{k_1, k_2 \geq 0} [\dots] = \max_{k_1 \geq 0} [k_1 h \nabla_+ V - C_1] + \max_{k_2 \geq 0} [k_2 h \nabla_- V - C_2]$$

So focus on one reaction. The problem for reaction 1 is:

$$\max_{k_1 \geq 0} \left[k_1 h \nabla_+ V - C_{\text{KL}}(k_1) \right]$$



Step 1: What the KL Cost Actually Looks Like for One Reaction

The KL cost for reaction 1 is (from Eq. 31, setting $f(x) = x \log x - x + 1$):

$$C_{\text{KL}}(k_1) = h(n_A) \left(k_1 \log \frac{k_1}{k_1^0} - k_1 + k_1^0 \right)$$

Let's check this makes sense:

- At $k_1 = k_1^0$: cost = $h(k_1^0 \log 1 - k_1^0 + k_1^0) = h \cdot 0 = 0 \checkmark$ no cost when uncontrolled
- At $k_1 \rightarrow 0$: cost $\rightarrow h \cdot k_1^0 > 0 \checkmark$ suppressing a reaction costs something
- At $k_1 \rightarrow \infty$: cost $\rightarrow \infty \checkmark$ you can't boost reactions for free
- Derivative at k_1^0 : $\frac{\partial C}{\partial k_1} \big|_{k_1^0} = h \log(k_1^0/k_1^0) = 0 \checkmark$ minimum at k_1^0

So the full optimization problem for reaction 1 is:

$$\max_{k_1 \geq 0} \left[k_1 h \nabla_+ V - h \left(k_1 \log \frac{k_1}{k_1^0} - k_1 + k_1^0 \right) \right]$$

Factor out h (it's positive so doesn't affect the max):

$$h \cdot \max_{k_1 \geq 0} \left[k_1 \nabla_+ V - k_1 \log \frac{k_1}{k_1^0} + k_1 - k_1^0 \right]$$

To maximize over k_1 , take derivative and set to zero:

$$\frac{d}{dk_1} \left[k_1 \nabla_+ V - k_1 \log \frac{k_1}{k_1^0} + k_1 - k_1^0 \right] = 0$$

Differentiate term by term:

$$\nabla_+ V - \frac{d}{dk_1} \left[k_1 \log \frac{k_1}{k_1^0} \right] + 1 = 0$$

For the middle term, use product rule:

$$\frac{d}{dk_1} \left[k_1 \log \frac{k_1}{k_1^0} \right] = \log \frac{k_1}{k_1^0} + k_1 \cdot \frac{1}{k_1} = \log \frac{k_1}{k_1^0} + 1$$

Substitute back:

$$\nabla_+ V - \log \frac{k_1}{k_1^0} - 1 + 1 = 0$$

$$\nabla_+ V = \log \frac{k_1}{k_1^0}$$

Exponentiate both sides:

$$k_1^\dagger = k_1^0 \cdot e^{\nabla_+ V} = k_1^0 \cdot e^{V(n_A+1) - V(n_A)}$$

Exponentiate both sides:

$$k_1^\ddagger = k_1^0 \cdot e^{\nabla_+ V} = k_1^0 \cdot e^{V(n_A+1)-V(n_A)}$$

Same calculation for reaction 2:

$$k_2^\ddagger = k_2^0 \cdot e^{\nabla_- V} = k_2^0 \cdot e^{V(n_A-1)-V(n_A)}$$

This is **Eq. 32** of the paper.

Step 10: Plug $k^\dagger$ Back Into the HJB

Now substitute the optimal rates back into $\max_k [L_k V - C]$.

For reaction 1, the optimized value (what we're maximizing evaluated at the optimum) is:

$$k_1^\dagger h \nabla_+ V - h \left(k_1^\dagger \log \frac{k_1^\dagger}{k_1^0} - k_1^\dagger + k_1^0 \right)$$

Substitute $k_1^\dagger = k_1^0 e^{\nabla_+ V}$ and $\log(k_1^\dagger/k_1^0) = \nabla_+ V$:

$$= k_1^0 e^{\nabla_+ V} h \nabla_+ V - h (k_1^0 e^{\nabla_+ V} \cdot \nabla_+ V - k_1^0 e^{\nabla_+ V} + k_1^0)$$

$$= h [k_1^0 e^{\nabla_+ V} \nabla_+ V - k_1^0 e^{\nabla_+ V} \nabla_+ V + k_1^0 e^{\nabla_+ V} - k_1^0]$$

$$= h \cdot k_1^0 [e^{\nabla_+ V} - 1]$$

So the full HJB with optimal control becomes:

$$-\frac{\partial V_t}{\partial t} = \beta U_t(n) + \sum_r k_r^0 h(n) [e^{\nabla_{s_r} V_t} - 1]$$

For Moran explicitly:

$$\boxed{-\frac{\partial V_t}{\partial t} = \beta U_t + k_1^0 h [e^{V(n_A+1)-V(n_A)} - 1] + k_2^0 h [e^{V(n_A-1)-V(n_A)} - 1]}$$

This is **Eq. 33** of the paper. Still nonlinear — because e^V is nonlinear in V .

Step 11: The Cole-Hopf Substitution — Killing the Nonlinearity

Define a new variable:

$$Z_t(n, \beta) := e^{V_t(n, \beta)}$$

so $V_t = \log Z_t$.

Now express everything in terms of Z :

$$\nabla_+ V = V(n_A + 1) - V(n_A) = \log Z(n_A + 1) - \log Z(n_A) = \log \frac{Z(n_A + 1)}{Z(n_A)}$$

Therefore:

$$e^{\nabla_+ V} = \frac{Z(n_A + 1)}{Z(n_A)}$$

and:

$$e^{\nabla_+ V} - 1 = \frac{Z(n_A + 1) - Z(n_A)}{Z(n_A)}$$

Also the time derivative:

$$\frac{\partial V_t}{\partial t} = \frac{\partial \log Z_t}{\partial t} = \frac{1}{Z_t} \frac{\partial Z_t}{\partial t}$$

Also the time derivative:

$$\frac{\partial V_t}{\partial t} = \frac{\partial \log Z_t}{\partial t} = \frac{1}{Z_t} \frac{\partial Z_t}{\partial t}$$

Substitute everything into Eq. 33 and multiply both sides by Z_t :

$$-\frac{\partial Z_t}{\partial t} = \beta U_t Z_t + k_1^0 h [Z_t(n_A + 1) - Z_t(n_A)] + k_2^0 h [Z_t(n_A - 1) - Z_t(n_A)]$$

The right side is just $\beta U_t Z_t + L_{k^0} Z_t$:

$$\boxed{-\frac{\partial Z_t}{\partial t} = \beta U_t(n) Z_t + L_{k^0} Z_t}$$

This is Eq. 34 of the paper. **Linear in Z .**

Step 12: The Feynman-Kac Formula — Computing Z Without Solving Any System

We have the linear equation (Eq. 34):

$$-\frac{\partial Z_t}{\partial t} = \beta U_t(n) Z_t + L_{k^0} Z_t$$

with terminal condition $Z_T(n) = e^{\beta U_T(n)}$.

Now I'll show you a completely different way to get the solution — not by solving equations, but by **running simulations**.

Step 13: What Does a Linear PDE Solution Look Like?

First let's build intuition with something you already know.

Consider the simplest linear ODE:

$$-\frac{dz}{dt} = \lambda z, \quad z(T) = z_T$$

Solution going **backward** from T :

$$z(t) = z_T \cdot e^{\lambda(T-t)}$$

Now our equation (Eq. 34) is similar but:

- λ is replaced by $\beta U_t(n)$ — state-dependent
- There's an extra term $L_{k^0} Z_t$ — the generator, which mixes states

The solution to this kind of equation has a beautiful probabilistic form. Let me derive it.

Step 14: Building the Probabilistic Representation

Run the **uncontrolled** process — meaning use rates k_1^0, k_2^0 with no control. Call the resulting trajectory $n^0(\tau)$ for $\tau \in [t, T]$.

Define a random variable along this trajectory:

$$M(\tau) := Z_\tau(n^0(\tau), \beta) \cdot \exp \left[\int_t^\tau \beta U_s(n^0(s)) ds \right]$$

Claim: $M(\tau)$ is a **martingale** under the uncontrolled process. That means:

$$\mathbb{E}_{k^0}[M(\tau') \mid n^0(\tau) = n] = M(\tau) \quad \text{for all } \tau' > \tau$$

A martingale is a process with no expected future drift — its expected future value always equals its current value. Like a fair game.

Why is M a martingale? Take the time derivative of M (using Itô's formula for jump processes):

$$\frac{dM}{d\tau} = \underbrace{\frac{\partial Z_\tau}{\partial \tau}}_{\text{smooth part}} \cdot e^{\int \beta U} + Z_\tau \cdot \underbrace{\beta U_\tau}_{\text{from the exp term}} \cdot e^{\int \beta U} + \underbrace{L_{k^0} Z_\tau \cdot e^{\int \beta U}}_{\text{jump part}}$$

$$= e^{\int \beta U} \left[\frac{\partial Z_\tau}{\partial \tau} + \beta U_\tau Z_\tau + L_{k^0} Z_\tau \right]$$

But Eq. 34 says exactly that the bracket = 0! So $dM/d\tau = 0$ in expectation — M is a martingale. ✓

Now the Full Chain Together

$$\underbrace{Z_t(n, \beta)}_{M(t) \text{ --- start}} = \mathbb{E}_{k^0} \left[\underbrace{e^{\beta U_{\text{total}}}}_{M(T) \text{ --- end}} \mid n^0(t) = n \right]$$

In words:

- At $\tau = t$: the integral $\int_t^t = 0$, so $M(t) = Z_t \cdot 1 = Z_t$
- At $\tau = T$: the integral covers the whole path, and $Z_T = e^{\beta U_T}$ by terminal condition, so $M(T) = e^{\beta U_{\text{total}}}$
- Martingale says these two are connected by expectation

Step 22: Why the Value Function Becomes Time-Independent

For a first-exit problem, ask yourself: if I'm in state n_A right now, does it matter what time it is?

No. The future dynamics depend only on n_A , not on the clock. The system has no memory of time — only memory of state. This is the Markov property.

So the value function must satisfy:

$$V_t(n, \beta) = V(n, \beta) \quad \text{for all } t$$

Time-independent. Which means:

$$\frac{\partial V_t}{\partial t} = 0$$

Step 23: HJB Collapses to a Static Equation

Recall the HJB (Eq. 33 after KL optimization):

$$-\frac{\partial V_t}{\partial t} = \beta U(n) + k_1^0 h [e^{\nabla+V} - 1] + k_2^0 h [e^{\nabla-V} - 1]$$

Set $\partial V_t / \partial t = 0$:

$$0 = \beta U(n) + k_1^0 h [e^{\nabla+V} - 1] + k_2^0 h [e^{\nabla-V} - 1]$$

Now apply Cole-Hopf $Z = e^V$ exactly as before. The same algebra gives (Eq. 37):

$$0 = \beta U(n) Z(n) + L_{k^0} Z(n)$$

with boundary conditions:

$$Z(0) = e^{\beta U_{\text{exit}}(0)} = e^0 = 1$$

$$Z(N) = e^{\beta U_{\text{exit}}(N)} = e^0 = 1$$

No time derivative anywhere. This is now just a **system of algebraic equations** — one equation per state.

Step 24: Writing It Out Explicitly

For each interior state $n_A \in \{1, \dots, N - 1\}$, expand $L_{k^0} Z$:

$$0 = \beta \cdot 1 \cdot Z(n_A) + k_1^0 h(n_A)[Z(n_A + 1) - Z(n_A)] + k_2^0 h(n_A)[Z(n_A - 1) - Z(n_A)]$$

where $U(n_A) = 1$ (earn 1 per unit time alive) and $h(n_A) = n_A(N - n_A)/N$.

Rearrange — collect terms by which Z they multiply:

$$\underbrace{[\beta - (k_1^0 + k_2^0)h]}_{\text{coefficient of } Z(n_A)} Z(n_A) + \underbrace{k_1^0 h}_{\text{coefficient of } Z(n_A+1)} Z(n_A + 1) + \underbrace{k_2^0 h}_{\text{coefficient of } Z(n_A-1)} Z(n_A - 1) = 0$$

For $N = 5$ as a small example, the states are $n_A \in \{1, 2, 3, 4\}$ and the system looks like:

$$\begin{pmatrix} \beta - (k_1 + k_2)h_1 & k_1^0 h_1 & 0 & 0 \\ k_2^0 h_2 & \beta - (k_1 + k_2)h_2 & k_1^0 h_2 & 0 \\ 0 & k_2^0 h_3 & \beta - (k_1 + k_2)h_3 & k_1^0 h_3 \\ 0 & 0 & k_2^0 h_4 & \beta - (k_1 + k_2)h_4 \end{pmatrix} \begin{pmatrix} Z(1) \\ Z(2) \\ Z(3) \\ Z(4) \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

The right side comes from moving the boundary terms ($Z(0) = 1$ and $Z(N) = 1$) to the right.

This is exactly what `solve_Z()` does. A tridiagonal $Ax = b$.

Step 25: What Does the Solution Tell You?

Once you solve for $Z(n_A)$:

1. Value function:

$$V(n_A) = \log Z(n_A)$$

This is the maximum expected reward (weighted exit time) starting from n_A .

2. Optimal rates:

$$k_1^\dagger(n_A) = k_1^0 \cdot \frac{Z(n_A + 1)}{Z(n_A)}, \quad k_2^\dagger(n_A) = k_2^0 \cdot \frac{Z(n_A - 1)}{Z(n_A)}$$

These are the rates you plug into the SSA. No time dependence — the controller just looks at the current state and applies these rates forever.

3. Mode-switching condition:

$$\frac{Z(n_A + 1)}{Z(n_A)} \approx 1 \iff k_1^\dagger \approx k_1^0 \quad (\text{OFF mode})$$

$$\frac{Z(n_A + 1)}{Z(n_A)} \gg 1 \iff k_1^\dagger \gg k_1^0 \quad (\text{ON mode})$$

Step 26: The β_c Story — Why the System Breaks

Rewrite the linear system as:

$$(\beta \cdot I_U + \Omega_{\text{non}}^0) Z_{\text{non}} = b$$

where Ω_{non}^0 is the generator matrix restricted to interior states, and I_U is diagonal with $U(n_A) = 1$.

For this to have a solution, the matrix $(\beta I + \Omega_{\text{non}}^0)$ must be **invertible**.

It becomes singular exactly when β is an eigenvalue of $-\Omega_{\text{non}}^0$.

The smallest such eigenvalue is $\beta_c = \lambda_1$.

What happens physically as $\beta \rightarrow \beta_c$?

$$Z(n_A) \rightarrow \infty \implies V(n_A) = \log Z \rightarrow \infty$$

And recall $V(n, \beta) = \log \mathbb{E}_{k^0}[e^{\beta T_{\text{exit}}}]$.

The moment generating function of T_{exit} diverges at β_c . This means the exit time distribution has an exponential tail — and β_c is the **rate of that tail**:

$$P(T_{\text{exit}} > t) \sim e^{-\beta_c t} \quad \text{for large } t$$

Controlled optimally at cost β_c per unit time, you can make the population survive **indefinitely on average**. Below β_c : extinction is inevitable. Above: you can't even achieve the target — the system is uncontrollable there.

Step 27: The Feynman-Kac for First-Exit

For the time-independent case, Feynman-Kac becomes:

$$Z(n, \beta) = \mathbb{E}_{k^0} \left[e^{\beta T_{\text{exit}}} \mid n(0) = n \right]$$

This is the moment generating function of the exit time under the **uncontrolled** dynamics. No control needed to evaluate it — just run SSA with k_1^0, k_2^0 , record T_{exit} for each run, compute $e^{\beta T_{\text{exit}}}$, average.

For the Moran process this has an **analytical form** (Eqs. 56-59 in the paper) using eigenvalues of the transition matrix. That's how the paper verifies the numerical `solve_Z()` solution.

Then What is β_c ?

β_c is the maximum value of β that makes mathematical sense.

Remember the linear system we solve:

$$\underbrace{(\beta \cdot I + \Omega_{\text{non}}^0)}_{\text{matrix } A} Z = b$$

For this to have a solution, matrix A must be invertible. It becomes singular when β equals an eigenvalue of $-\Omega_{\text{non}}^0$.

The smallest such eigenvalue is β_c .

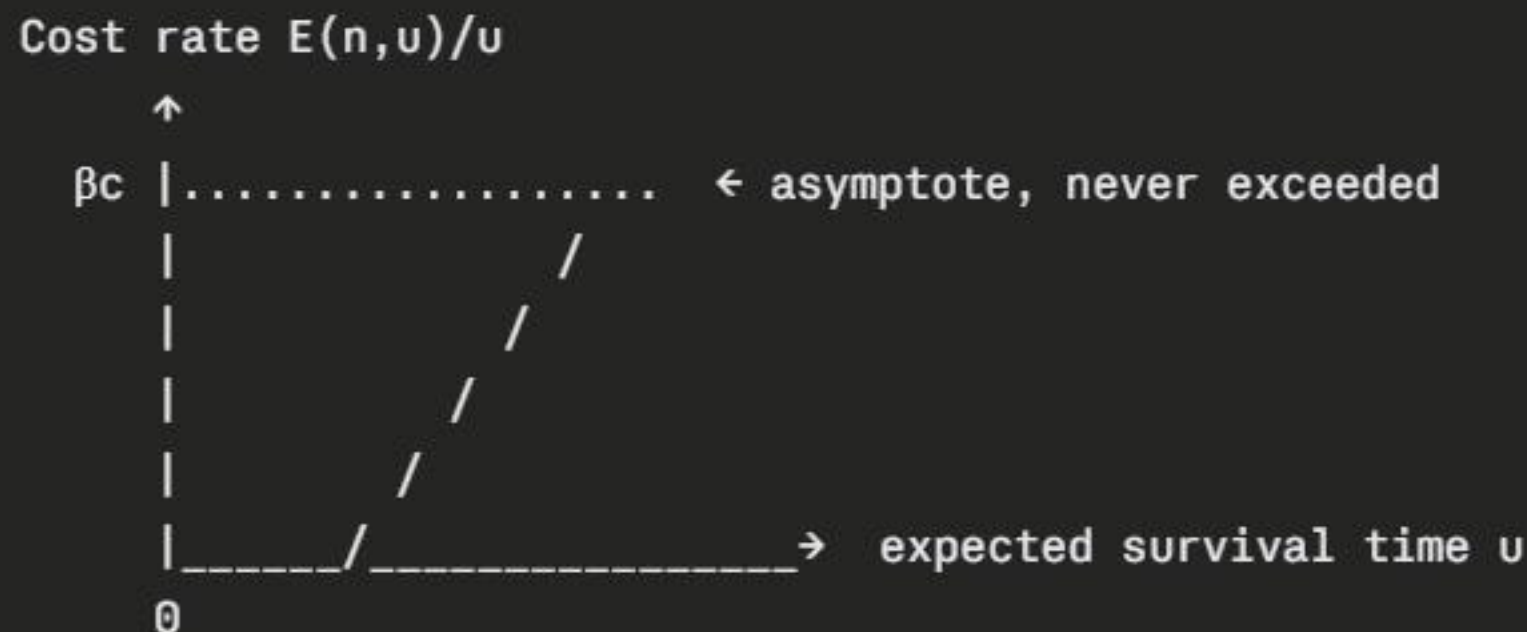
So:

```
 $\beta < \beta_c \rightarrow$  matrix  $A$  is invertible  $\rightarrow Z$  finite  $\rightarrow$  solution exists  
 $\beta = \beta_c \rightarrow$  matrix  $A$  is singular  $\rightarrow Z \rightarrow \infty \rightarrow$  no solution  
 $\beta > \beta_c \rightarrow$  matrix  $A$  has wrong sign  $\rightarrow Z$  negative  $\rightarrow$  nonsense
```


Physical Meaning of β_c

Here is the deep meaning — the minimum cost rate you must pay to prevent extinction indefinitely.

Think of it this way. The cost-survival tradeoff curve from the paper (Fig 4c):



If you pay cost rate $< \beta_c$ per unit time $\rightarrow$ population eventually goes extinct regardless.

If you pay cost rate $= \beta_c$ per unit time $\rightarrow$ population survives indefinitely on average.

Paying more than β_c does not help — you already achieved infinite survival at exactly β_c .

So β_c is a **phase transition point** — below it, extinction is inevitable. At it, survival becomes possible forever.

So What is the Actual Relationship Between β and β_c ?

They answer completely different questions:

	β	β_c
What is it?	Your choice of how much you value survival	A property of the biological system
Who sets it?	You, externally	Computed from k_1^0, k_2^0, N
What happens at it?	Nothing special	Value function diverges
Physical meaning	Control strength knob	Minimum cost to prevent extinction
In the code	Swept over many values	Used to bound the sweep safely

Concrete Example

Say $\beta_c = 0.47$ (like $\gamma = 1.5$ in the code).

You might run the experiment with $\beta \in \{0.05, 0.10, 0.20, 0.40, 0.44\}$ — all below β_c .

Each β gives a different controller — progressively more aggressive as β increases.

At $\beta = 0.44 \approx 0.95 \cdot \beta_c$: controller is working almost at maximum capacity.

At $\beta = 0.47 = \beta_c$: mathematically the controller achieves infinite expected survival, but $Z \rightarrow \infty$ and the solver breaks numerically.

At $\beta = 0.50 > \beta_c$: nonsense — no valid solution exists.